Strategic Indexing Techniques for Taxi Trajectory Query Optimization

Ruitao Sun

***Abstract*—Efficiently retrieving data from databases consisting of location and time-specific details is growing increasingly significant for systems involving traffic management. This report explores uses of indexing technicques—like GiST (Generalizd Search Tree), SP-GiST (Space Partitioned GiST), BRIN (Block Range Index), Hash Indexing, and Btree Indexing—within PostgreSQL frameworks. These methods will be assessed for their effectiveness on improving the time of execution and accuracy of queries relating to taxi Trajectories.**

This study focuses on query tasks; such as optimizing queries for finding the most similar trajectory, and identifying k nearest neighbors (data points) of a given trajectory for a given date. Also, determining skyline data points for certain days. Through testing and performance analyzing by each indexing technique, the aim is to gain an understanding of how different indexing strategies can be optimized for managing complex queries involving spatial, and temporal data.

The experiment reveals variations at the performance of each indexing technical, shedding light to when each method excels or falls short. [1] These insights are important for database administrations and developers working with high dimensional temporal datasets, Furthermore, this research contributes to discussion about enhancing database technologies. For efficiency and accuracy in handling data manipulation and retrieval tasks.

The results not only help us grasp real-world uses of trajectory data and constraints of indexing methods but also set the stage for upcoming advancements in managing database systems.

I. INTRODUCTION

FOR the modern cities, dealing with taxi trajectory data is crucial not only for taxi companies but also for helping cities manage traffic and better plan urban environments. The taxi trajectory dataset [2] for this report covers a full year of taxi data in Porto, a city of Portugal, which are tracked down by GPS on 442 taxis. It includes information such as each taxi's ID, trip times, and how passengers hailed the taxis—whether through a call center, at a taxi stand, or by flagging one down on the street.

There are some practical benefits of using this data. Considering the taxi companies, it will help taxi services become faster and also reduce the amount of gas consumption, which as a result, makes the taxi hail cheaper and less polluting. Traffic managers can use this data to determine where the traffic gets congested the most, which helps the company make better decisions on where to build new roads or bus routes. This could also help reduce car pollution, making the city a better place to live.

In order to handle the large amount of data more efficiently, there is a need to use cutting-edge database techniques like indexing. The index helps the database easily locate and utilize the data without needing to look everything through, which is important when we request information in real time, like calculating and setting taxi fares or dispatching taxis. Some types of indexing, such as GiST (Generalized Search Tree) and BRIN (Block Range Index), perform quite well for working with the kind of location data in the dataset.

This data is not just playing a significant role for businesses, researchers could also use the data to develop modern ways for traffic management, use artificial intelligence to predict traffic conditions, and investigate how different factors such as time of day or the way taxis are hailed can potentially affect traffic. By making it easier to play around with the data, we can speed up research and start applying new ideas in the future.

In general, with the taxi trajectory data, we can help make taxis hail more efficient, also make the cities well organized, and give us new insights into how people move around in urban areas. By improving how we search and use this data, we can make some real changes that make our city a better place to live.

II. METHODOLOGY

A. Preprocessing

For the preprocessing stage, it is significant to guarantee the usability and quality of the data for further analysis. The first objective for this stage is to reduce any duplication and ensure the data completeness:

- **Drop Duplication**: The Python library: Pandas, is used for the first step includes loading the CSV file and dropping the duplicate rows afterwards. This process ensures that unique data points are further processed.
- **Dealing with Missing Data**: Data Point with missing GPS records, indicated by the MISSING_DATA column as True, are filtered out. This step is important since incomplete data could potentially distort the trajectory analysis and somehow downstream applications like route optimization or traffic pattern analysis.
- **Getting Unique Trip ID**: Next step is to ensure that each trip is uniquely existed. Non-duplicate TRIP_ID makes sure the success of importing the TRIP_ID as primary key. The trajectory dataset is once again filtered to remain only the first occurrence of each trip identifier, dropping

subsequent duplicates that might have not been filtered out during earlier cleaning stages. This ensures integrity of data, with each trip distinctly identified by its TRIP_ID.

- **Data Export**: The cleaned and non-duplicated dataset is then saved to another CSV file naming taxi.csv, preparing for data ingestion in the next step.

B. Data Ingestion

For the data ingestion, the ready-to-use CSV file is ingested into a PostgreSQL database to ensure efficient query processing:

- **Defining the Schema**: project.taxi table is created with a schema closely reflecting the dataset's structure. This includes fields like TRIP_ID, CALL_TYPE, ORIGIN_CALL, ORIGIN_STAND, TAXI_ID, TIMESTAMP, DAY_TYPE, MISSING_DATA, and POLYLINE, each field and data type is designed in a way to align with each other in the dataset.
- **Importing the Data**: The CSV file is ingested into a newly created table using PostgreSQL's native bulk copy command (\copy). This command is optimized for large data volumes, ensuring rapid data ingestion while following the defined schema and data types.

C. Data Querying and Geometry Handling

After data ingestion, we are focusing on query and spatial analysis: Geometry POLYLINE data, which is in JSON format representing sequences of GPS coordinates, is converted into a geometric line type using PostGIS functions. This includes parsing the POLYLINE JSON data to extract the latitude and longitude. All the converted data is stored in a newly created column named polyline_geom. And the using parsed coordinates to create points using ST_MakePoint and aggregating these points into a line using ST_MakeLine. In the meantime, setting the spatial reference identifier (SRID) to ensure accurate geographic positioning.

D. Query Task: Finding the Most Similar Taxi Trajectory

The objective of this task is to identify the taxi trajectory that has the greatest similarity to a trajectory specified by a TRIP_ID. The query includes calculating the geometric distances between the reference trajectory and all other trajectories within the dataset, and eventually identifying the data with the minimum distance.

A WITH clause (ReferenceTrajectory) is used to isolate the polyline geometry of the taxi trajectory specified by TRIP_ID = 1372636951620000320. We can use Reference Trajectory for further comparison.

And then Using the ST_DWithin function, the query filters out trajectories that are not within a threshold distance of 0.01 from the reference trajectory. The step reduces the computational burden by limiting the number of trajectories that need to be used in further calculations. The application of spatial indexes (GiST, BRIN, SP-GiST) on the polyline_geom column significantly accelerates this spatial query, enhancing performance by facilitating fast distance checks.

1) **Computing the distance:**

- Hausdorff Distance [3]: The ST_HausdorffDistance function is applied to compute the max distance between the reference trajectory and all the other filtered trajectories, measured by the similarity where a smaller value indicates greater similarity.
- Sort and Limit the result: The result is sorted by ascending order, and the top trajectory with the smallest Hausdorff distance to the reference is returned. This ensures that the trajectory with the greatest similarity is retrieved correctly.

2) **Indexing Methods**

- GiST (Generalized Search Tree):

Gist is applied to polyline_geom field to facilitate the performance of ST_DWithin and ST_HausdorffDistance operations.

- BRIN (Block Range Index):

BRIN indexing can be super-efficient for large datasets where data are highly related and stored in a way that is physically close to each other. This can be useful for certain similarity calculation queries where data has a related distribution.

- SP-GiST (Space-Partitioned GiST):

SP-GIST is an extension of GiST, optimizing the partitioning space into non-overlapping regions.
It can potentially be very useful for checking distances between non-overlapping geometries.

3) **Improvements and Innovations**

- The query is constructed in a way that only contains the trajectory with less distance. In this way, there will be less compared trajectories with referenced trajectory, which will reduce the computational cost. Also, using different types of index will ensure that the database functionality is fully used to speed the query execution.

E. Query Task: Finding k Nearest Neighbors of a Given Trajectory on a Specific Date

The aim of this task is using spatial query and indexing techniques to efficiently find the k nearest neighbor trajectories of a specified taxi trajectory on a given date. This includes determining the proximity of taxi trajectories among each other, leveraging both the geometrical data stored in the polyline_geom and optimizing queries through specific PostgreSQL indexing methods.

The query begins with choosing a reference trajectory using a given trip_id and storing its geometry temporarily in a Common Table Expression (CTE) naming SelectedTrajectory. The trajectory serves as a benchmark for distance calculations. The other CTE, called filteredTrajectory, uses the ST_DWithin function to efficiently identify nearby trajectories within a 0.01 threshold, which will minimize computational

cost. The real distances between the reference trajectory and those close ones are calculated using the ST_Distance function. And then, setting the query to a predefined specific date ('2014-02-15'), enhancing relevance. Spatial indexes like GiST, SP-GiST, and BRIN are applied to improve execution performance. In the end, the results are ordered by distance in ascending order, optimizing the retrieval of the closest trajectories, crucial for applications like dynamic routing. This method combines pre-filtering and precise measurements, leveraging multiple indexing strategies for enhanced performance.

F. Query Task: Identify skyline data points for a certain day.

The goal of this task is to find the skyline points for a specific day, sky points in this scenario are defined as those trips when compared with all the other existing trips, that are shorter in time and equal or longer in distance, or equal in time and longer in distance. This definition ensures that the result includes only the trips that are dominant in terms of less time or greater distance.

1) Add Columns

- Introducing only geometry data in the skyline data for this task seems impossible. The initial test was applied when the polyline_geom is not converted in terms of time or distance. My initial test without data type conversion took 2 hours with no result.
- Thinking about the possible solution, we can simply break down the spatial data to trajectory distance and travel time.
- The initial step is to convert GEOMETRY type into GEOGRAPHY type and then apply ST_LENGTH to calculate the distance for each trajectory.
- The next step is to calculate the distance using the formula provided by Kaggle. The distance is calculated by 15 times the number of data points minus 1. (start point) This way the trajectory is highly optimized for further query.

2) SQL Query for Skyline Points:

- Step 1 is to use a CTE to isolate the distance of a certain trajectory on a certain day (defined in test cases)
- Then use a self-left join to compare each trip against all others based on the defined skyline criteria.
- Trips that do not find any matching criteria in the join (indicating they are not dominated by any other trip) are selected as skyline points.

3) Indexing for Performance Optimization:

- HASH Index
- BRIN Index:
- BTREE Index

4) Innovation and improvement

- Geographical Calculations: The conversion from geometry to geography types ensures that the distance calculations are accurate over the geographic surface, adding two columns make the query faster by transforming geospatial data to normal float data.

III. Experimental Results & Analysis

A. Query Task 1: Trajectory Similarity

For query trajectory similarity, we applied three different indexing methods (GiST, SP-GiST, and BRIN) and compared each index performance to find the most similar trajectory to a given trajectory in the dataset.

Performance Analysis

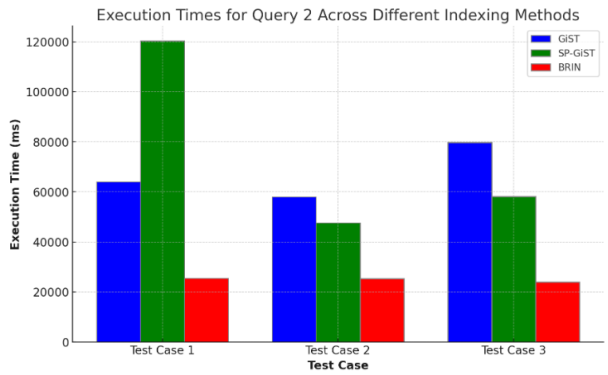
- ***GiST:*** execution time varies a lot for 3 different test cases, which means GIST is sensitive to data distribution or complexity of the spatial query condition. It performs somehow slow for the very first test case, however, it shows great improvement in simpler tests such as test case 3.
- ***SP-GiST:*** Changes greatly for different test cases, but less significant than GiST. Its performance are ranked as number 2 out of all 3 test cases, suggesting it is moderate efficiency across different data conditions.
- ***BRIN:*** Execution time does not change a lot and always return lower execution times, suggesting it is more suitable for trajectory similarity analysis.

Conclusion

Generally speaking, BRIN performs the best among the 3 according to the query plan result. It shows a significant reduction in execution for test case 1 compared to another 2 indexing methods. BRIN shows potential for consistent performance across tested conditions, due to its efficiency in handling ordered data. However, depending on the data's spatial characteristics and the specific requirements of the query, GiST or SP-GiST sometimes could provide better performance.

B. Query Task 2: KNN for a given Trajectory on a certain day.

For the KNN for given trajectory, three different indexing methods: GiST, SP-GiST, and BRIN.[4] are used. The query utilizes spatial functions to filter and sort data based on proximity to a specified trajectory.



Performance Analysis

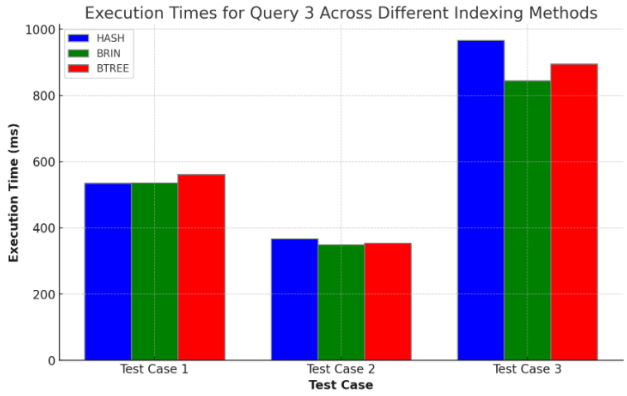
- **GiST:** The test result shows that GIST execution time is similar for all 3 test cases, meaning that it could potentially be useful with the query is complex.
- **SP-GiST:** Varies to a great extent, it performs super badly in the first test case and moderately for test case 2 and 3
- **BRIN:** Consistently performs the best among all test cases, indicating strong performance in scenarios where data has inherent ordering or the query extensively benefits from block-level summarization.

Conclusion

Obviously, BRIN has consistently superior performance. Compared with the previous result in query1, BRIN also has one of the best results. This suggests that in real-world scenarios involving large-scale geo-temporal data, such as this taxi dataset, BRIN indexing maintaining some form of ordering or block summarization can drastically improve query efficiency.

C. Query Task 3: Skyline Data Points

Three different indexing methods HASH, BRIN, and BTREE—are evaluated for their performance.



This query task is a bit different from the previous two queries. The spatial data polyline_geom is converted from geometry to geography and then calculated in meters. Similarly, travel time is calculated as an integer. Since there is a complex use of timestamp conversion, the three indexes HASH, BRIN, and BTREE are applied.

Performance Analysis

- Based on the performance of all three indexes, they are quite similar regarding the execution time. They vary according to the query structure. Taking a deep analysis, previously applying data transformation from spatial data to travel time as integer and distance as float which makes the complex skyline data point way easier regarding computational cost.

Conclusion

The efficiency indexing methods in processing complex queries like skylines is highly dependent on the data's characteristics and the query structure. In this scenario, the performance of three different index methods is quite similar, meaning that conversion from spatial data to numerical data makes the performance of the index less significant.

V. Conclusion

Indexing taxi trajectory data properly will make taxis run in a more efficient way. It can save time for taxi hail, avoid and reduce traffic congestion, and subsequently reduce car pollution. This study guides people to index not just in theory but directly facilitate the urban transportation system.

References

[1] L. O. Alvares, V. Bogorny, and B. Kuijpers, "A Model for Enriching Trajectories with Semantic Geographical Information," ResearchGate, [Online].Available:https://www.researchgate.net/publication/221589752_A_Model_for_Enriching_Trajectories_with_Semantic_Geographical_Information.

[2] Crailtap, "Taxi Trajectory," Kaggle, 2021. [Online]. Available: https://www.kaggle.com/datasets/crailtap/taxi-trajectory

[3] PostGIS, "ST_HausdorffDistance," PostGIS 2.5 Documentation. [Online]. Available: https://postgis.net/docs/ST_HausdorffDistance.html.

[4] Citus Data, "A Tour of Postgres Index Types," Oct. 17, 2017. [Online]. Available: https://www.citusdata.com/blog/2017/10/17/tour-of-postgres-index-types/.